My CheatSheet: C Programming Language

1. Comments & Basics....2
2. Compile & Run....2
3. Variables & Constants....2
4. Input & Output....5
5. If & Switch....7
6. While & For....8
7. Arrays....9
8. Functions & Strings....10
9. Pointers & Static....16
10. Memory Management....17
11. Structures....18
12. Unions....19
13. Enumerations....20
14. Files....20
15. Math Functions....22
16. Random....22
17. Date-Time & Sleep....23
18. Preprocessor Directives....24
19. Argc & Argv....26
20. Multithreading....26
21. Cool Print....28
22. Execution Time....31
23. Bonus: Linux....32

1. Comments & Basics

```
// Single-line comment

/*
Multi-line
comment
*/

// printf:
#include <stdio.h>

int main(void)
{
    printf("Hello world!\n");

    return 0;
}
```

2. Compile & Run

```
# Compile:
gcc -std="c99" -Wall -Wextra -Wpedantic -fno-builtin -O0 "code.c" -o "program_c"

# Run:
./"program_c"

# C to Assembly:
# gcc -std="c99" -Wall -Wextra -Wpedantic -fno-builtin -O0 -S -masm=intel "code.c" -o "code.s"

# #include <math.h>:
# gcc -std="c99" -Wall -Wextra -Wpedantic -fno-builtin -O0 "code.c" -o "program_c" -lm

# #include <pthread.h>:
# gcc -std="c99" -Wall -Wextra -Wpedantic -fno-builtin -O0 "code.c" -o "program_c" -pthread
```

3. Variables & Constants

```
// printf:
#include <stdio.h>

// bool, false, true:
#include <stdbool.h>

// Preprocessor constant char:
#define LETTER 'C'

// Preprocessor constant bool:
#define ACTIVE true

// Preprocessor constant int:
#define NUMBER -1

// Preprocessor constant unsigned int:
#define UNSIGNED_NUMBER 10u

// Preprocessor constant float:
// #define PI 3.14159f
```

```
// Preprocessor constant double:
#define PI 3.14159

// Preprocessor constant string:
#define TEXT "Some example"

// Avoid using global variables when possible:
int global_number = 10;

int main(void)
{
    // 1 byte:
    char c = 'A';
    // char c = 65;

    // 1 byte:
    bool b = 1;
    // 0 = false; non-zero = true

    // 4 bytes:
    int i = -100;
    // signed (by default) = negative and non-negative values
    // int ascii = 'A';

    // 4 bytes:
    unsigned int ui = 100u;
    // unsigned = non-negative values only
    // Custom alias:
    // typedef unsigned int uint;
    // uint ui = 100u;

    // 4 bytes:
    float f = 99.99f;

    // 8 bytes:
    double d = 99.99;

    // 7 bytes = 6 chars + '\0' char:
    char s[] = "Hello!";
    // char s[] = {'H', 'e', 'l', 'l', 'o', '!', '\0',};

    // More data type modifiers:
    // short
    // long
    // long long

    // ------------------------------------------------------------

    printf("Char: %c\n", c);
    printf("Bool: %d\n", b);
    printf("Int: %d\n", i);
    printf("Int: %9d\n", i);
    printf("Int: %-9d\n", i);
    printf("Global variable: %d\n", global_number);
    printf("Unsigned int: %u\n", ui);
    printf("Octal: %o\n", ui);
    printf("Octal: \101\n");
    printf("Hexadecimal lowercase: %x\n", ui);
    printf("Hexadecimal uppercase: %X\n", ui);
    printf("Hexadecimal: \x41\n");
    printf("Float: %f\n", f);
```

```
printf("Float: %.2f\n", f);
printf("Double: %lf\n", d);
printf("Double: %.2lf\n", d);
printf("String: %s\n", s);
// printf("Percent: %%\n");

printf("\n");

// ------------------------------------------------------------

printf("Char: %zu byte\n", sizeof(c));
printf("Bool: %zu byte\n", sizeof(b));
printf("Int: %zu bytes\n", sizeof(i));
printf("Unsigned int: %zu bytes\n", sizeof(ui));
printf("Float: %zu bytes\n", sizeof(f));
printf("Double: %zu bytes\n", sizeof(d));
printf("String: %zu bytes\n", sizeof(s));

printf("\n");

// ------------------------------------------------------------

// Constants:
const char TYPE = '@';
const bool HARD = 0;
const int MAX = 1000;
const unsigned int MIN = 0u;
// const float REAL = 8.3f;
const double REAL = 8.3;
const char WORD[] = "Test";

printf("Constant char: %c\n", TYPE);
printf("Constant bool: %d\n", HARD);
printf("Constant int: %d\n", MAX);
printf("Constant unsigned int: %u\n", MIN);
// printf("Constant float: %f\n", REAL);
printf("Constant double: %lf\n", REAL);
printf("Constant string: %s\n", WORD);

printf("Preprocessor constant char: %c\n", LETTER);
printf("Preprocessor constant bool: %d\n", ACTIVE);
printf("Preprocessor constant int: %d\n", NUMBER);
printf("Preprocessor constant unsigned int: %u\n", UNSIGNED_NUMBER);
// printf("Preprocessor constant float: %f\n", PI);
printf("Preprocessor constant double: %lf\n", PI);
printf("Preprocessor constant string: %s\n", TEXT);

printf("\n");

// ------------------------------------------------------------

// Data casting: ↓
int x = 10;
// Implicit casting (automatic):
double y = x;
printf("%lf\n", y);

double z = 9.9;
// Explicit casting (manual):
int k = (int) z;
printf("%d\n", k);
```

```
    return 0;
}
```

4. Input & Output

```
// printf, scanf, getchar, EOF, fgets, stdin:
#include <stdio.h>

// bool:
#include <stdbool.h>

// strcspn:
#include <string.h>

// NULL:
#include <stddef.h>

int main(void)
{
    // Escape sequences:
    // "\n"
    // "\r"
    // "\f"
    // "\t"
    // "\v"
    // "\a"
    // "\b"
    // "\\"
    // "\'"
    // "\""
    // "\?"
    // "\0"

    char c = '\0';
    bool b = 0;
    int ib = 0;
    int i = 0;
    unsigned int ui = 0u;
    float f = 0.0f;
    double d = 0.0;
    char s[100u + 1u];

    // ------------------------------------------------------------

    printf("Char input: ");
    if (scanf(" %c", &c) != 1)
    {
        printf("Invalid input!\n");
        return 1;
    }

    printf("Bool input: ");
    if (scanf("%d", &ib) != 1)
    {
        printf("Invalid input!\n");
        return 1;
    }
    b = ib;
```

```
printf("Int input: ");
if (scanf("%d", &i) != 1)
{
    printf("Invalid input!\n");
    return 1;
}

printf("Unsigned int input: ");
if (scanf("%u", &ui) != 1)
{
    printf("Invalid input!\n");
    return 1;
}

printf("Float input: ");
if (scanf("%f", &f) != 1)
{
    printf("Invalid input!\n");
    return 1;
}

printf("Double input: ");
if (scanf("%lf", &d) != 1)
{
    printf("Invalid input!\n");
    return 1;
}

// ------------------------------------------------------------

// After scanf() and before fgets():
while (1)
{
    int ch = getchar();

    if (ch == '\n' || ch == EOF)
    {
        break;
    }
}

// ------------------------------------------------------------

printf("String input: ");
if (fgets(s, sizeof(s), stdin) == NULL)
{
    printf("Invalid input!\n");
    return 1;
}
// Replace newline:
s[strcspn(s, "\n")] = '\0';

printf("\n");

// ------------------------------------------------------------

printf("Char: %c\n", c);
printf("Bool: %d\n", b);
printf("Int: %d\n", i);
printf("Unsigned int: %u\n", ui);
printf("Float: %f\n", f);
```

```
    printf("Double: %lf\n", d);
    printf("String: %s\n", s);

    return 0;
}
```

5. If & Switch

```
// printf:
#include <stdio.h>

// strcpy:
#include <string.h>

int main(void)
{
    // Assignment: =, +=, -=, *=, /=, %=, |=, &=, ^=, >>=, <<=
    // Arithmetic: +, -, *, /, %
    // Bitwise: |, &, ^, >>, <<, ~
    // Comparison: ==, !=, >, <, >=, <=
    // Logical: !, ||, &&
    // Ternary: ?, :
    // Increment: ++i or i++ (i += 1)
    // Decrement: --i or i-- (i -= 1)

    char s[13u + 1u];
    int i = -100;

    // Ternary (if_else):
    strcpy(s, (i < 0) ? "Negative" : "Not negative");
    printf("%s\n", s);

    printf("\n");

    // ------------------------------------------------------------

    // if:
    if (i < 0)
    {
        printf("Negative\n");
    }

    // if_else:
    if (i < 0)
    {
        printf("Negative\n");
    }
    else
    {
        printf("Not negative\n");
    }

    // if_else-if:
    if (i < 0)
    {
        printf("Negative\n");
    }
    else if (i == 0)
    {
        printf("Neutral\n");
```

```
    }
    else if (i > 0)
    {
        printf("Positive\n");
    }

    // Nested if:
    int age = 20;
    if (age >= 18)
    {
        printf("You are an adult\n");

        if (age >= 21)
        {
            printf("You can enter the club\n");
        }
    }

    printf("\n");

    // ------------------------------------------------------------

    // switch:
    char c = 'C';

    switch (c)
    {
        case 'B':
            printf("Bash\n");
            break;
        case 'C':
            printf("C\n");
            break;
        case 'J':
            printf("Java\n");
            break;
        case 'P':
            printf("Python\n");
            break;
        default:
            printf("Not my business!\n");
    }

    return 0;
}
```

6. While & For

```
// printf:
#include <stdio.h>

int main(void)
{
    // while:
    int i = 0;
    while (i < 10)
    {
        printf("%d\n", i);
        i += 1;
```

```
        // break;
        // continue;
        // goto label;
    }

    // label:
        // printf("Jump\n");

    // do-while:
    // do
    // {
        // printf("Done\n");
    // }
    // while (0);

    printf("\n");

    // ------------------------------------------------------------

    // for:
    for (int i = 0; i < 10; i += 1)
    {
        printf("%d\n", i);
    }

    // Nested for:
    for (int i = 0; i < 10; i += 1)
    {
        for (int j = 0; j < 10; j += 1)
        {
            printf("- ");
        }

        printf("\n");
    }

    return 0;
}
```

7. Arrays

```
// printf:
#include <stdio.h>

// size_t:
#include <stddef.h>

int main(void)
{
    // Array:
    int array[] = {6, 2, 3, 4, 5,};

    // Array size:
    size_t array_size = sizeof(array) / sizeof(array[0u]);
    printf("Array size: %zu\n", array_size);

    // Change value:
    array[0u] = 1;
    // printf("First element: %d\n", array[0u]);
```

```
    // Loop through:
    for (size_t i = 0u; i < array_size; i += 1u)
    {
        printf("%d\n", array[i]);
    }

    printf("\n");

    // ------------------------------------------------------------

    // 2D array:
    int matrix[3u][3u] =
    {
        {1, 2, 3,},
        {4, 5, 6,},
        {7, 8, 9,},
    };

    size_t rows = sizeof(matrix) / sizeof(matrix[0u]);
    size_t cols = sizeof(matrix[0u]) / sizeof(matrix[0u][0u]);

    for (size_t i = 0u; i < rows; i += 1u)
    {
        for (size_t j = 0u; j < cols; j += 1u)
        {
            printf("%d ", matrix[i][j]);
        }

        printf("\n");
    }

    printf("\n");

    // ------------------------------------------------------------

    // Array of strings:
    char fruits[2u][6u + 1u] = {"Apple", "Banana",};
    // 2 = number of strings; 7 = maximum capacity of each string

    printf("%s\n", fruits[0u]);

    return 0;
}
```

8. Functions & Strings

```
// printf, snprintf:
#include <stdio.h>

// va_list, va_start, va_arg, va_end:
#include <stdarg.h>

// String functions:
#include <string.h>

// Ctype functions:
#include <ctype.h>

// size_t, NULL:
#include <stddef.h>
```

```c
// atoi, strtof, strtod:
#include <stdlib.h>

// Function declaration:
double calculate_sum(double a, double b);
unsigned int calculate_factorial(unsigned int n);
void variadic_sum(size_t n, ...);
void swap_numbers(int *a, int *b);
void string_upper(char *s);
void string_lower(char *s);
void string_reversed(const char *s);
void array_sorted(int *d, size_t d_size, char c);
void print_array(const int *d, size_t d_size);
void convert_case(char *s, char c);
void swap_array(int *d, size_t j);

int main(void)
{
    // Function call: ↓
    double sum = calculate_sum(3.3, 3.4);
    printf("%lf\n", sum);

    unsigned int fac = calculate_factorial(5u);
    printf("%u\n", fac);

    variadic_sum(3u, 1.5, 2.5, 6.5);
    variadic_sum(4u, 1.5, 2.5, 6.5, 9.5);

    int x = 10;
    int y = 20;
    printf("Before swap: x = %d | y = %d\n", x, y);
    swap_numbers(&x, &y);
    printf("After swap: x = %d | y = %d\n", x, y);

    char name[] = "Nemanja";
    string_upper(name);
    string_lower(name);
    string_reversed(name);

    int array[] = {2, 1, 4, 3, 6, 5, 8, 7, 10, 9,};
    size_t array_size = sizeof(array) / sizeof(array[0u]);

    array_sorted(array, array_size, 'A');
    print_array(array, array_size);

    array_sorted(array, array_size, 'D');
    print_array(array, array_size);

    printf("\n");

    // ------------------------------------------------------------

    // String functions: ↓
    // Enough space for strcpy() and strcat():
    char str1[7u + 1u] = "Hello";
    char str2[] = "!";

    // Length 5:
    printf("Length %zu\n", strlen(str1));
```

```
// Length 1:
printf("Length %zu\n", strlen(str2));

// Size 8:
printf("Size %zu\n", sizeof(str1));

// Size 2:
printf("Size %zu\n", sizeof(str2));

// Compare:
if (strcmp(str1, str2) == 0)
{
    printf("Equal");
}

// Copy (overwrite):
strcpy(str1, "Better");

// Concatenate (append str2 to the end of str1):
strcat(str1, str2);

printf("%s\n", str1);

// Other string functions:
// strchr(str, 'A');
// strrchr(str, 'A');
// strstr(str, "sub");
// strtok(str, ", ");
// strcspn(str, ", ");
// strncpy(dest, src, n);
// strncat(dest, src, n);
// strncmp(str1, str2, n);
// ...

int i = 0;
float f = 0.0f;
double d = 0.0;
char s[15u + 1u];

// Convert string to int:
i = atoi("12345");

// Convert string to float:
f = strtof("12345", NULL);

// Convert string to double:
d = strtod("12345", NULL);

// Convert non-string to string:
int s_length = snprintf(s, sizeof(s), "Hello world%c x%d", '!', 3);

// Calculate the required formatted string length without storing the string, excluding the '\0':
// int s_length = snprintf(NULL, 0, "Hello world%c x%d", '!', 3);

if (s_length < 0)
{
    printf("Formatting error!\n");
    return 1;
}
else if ((size_t) s_length >= sizeof(s))
{
```

```
        printf("Output was truncated!\n");
        return 1;
    }

    printf("%d\n", i);
    printf("%f\n", f);
    printf("%lf\n", d);
    printf("%s\n", s);
    printf("%d\n", s_length);

    return 0;
}

// ------------------------------------------------------------

// Function definition: ↓
double calculate_sum(double a, double b)
{
    return a + b;
}

// Recursive function (result wraps around on unsigned int overflow):
unsigned int calculate_factorial(unsigned int n)
{
    if (n < 2u)
    {
        return 1u;
    }

    return n * calculate_factorial(n - 1u);
}

// Variadic function (double parameters in this example):
void variadic_sum(size_t n, ...)
{
    va_list args;
    va_start(args, n);

    double sum = 0.0;

    for (size_t i = 0u; i < n; i += 1u)
    {
        double d = va_arg(args, double);
        sum += d;
    }

    va_end(args);

    printf("Sum: %lf\n", sum);
}

void swap_numbers(int *a, int *b)
{
    if (a == NULL || b == NULL)
    {
        return;
    }

    int temp = *a;
    *a = *b;
    *b = temp;
```

```
}

void string_upper(char *s)
{
    convert_case(s, 'U');
}

void string_lower(char *s)
{
    convert_case(s, 'L');
}

void string_reversed(const char *s)
{
    if (s == NULL)
    {
        return;
    }

    size_t s_length = strlen(s);
    char temp[s_length + 1u];

    for (size_t i = 0u; i < s_length; i += 1u)
    {
        temp[i] = s[s_length - 1u - i];
    }

    temp[s_length] = '\0';
    printf("%s\n", temp);
}

void array_sorted(int *d, size_t d_size, char c)
{
    if (d == NULL || d_size < 2u)
    {
        return;
    }

    if (c != 'A' && c != 'D')
    {
        printf("Invalid sorting option (use 'A' or 'D')!\n");
        return;
    }

    for (size_t i = 0u; i < d_size - 1u; i += 1u)
    {
        for (size_t j = 0u; j < d_size - 1u - i; j += 1u)
        {
            if (c == 'A')
            {
                if (d[j] > d[j + 1u])
                {
                    swap_array(d, j);
                }
            }
            else if (c == 'D')
            {
                if (d[j] < d[j + 1u])
                {
                    swap_array(d, j);
                }
```

```
            }
        }
    }
}

void print_array(const int *d, size_t d_size)
{
    if (d == NULL || d_size == 0u)
    {
        return;
    }

    for (size_t i = 0u; i < d_size; i += 1u)
    {
        printf("%d ", d[i]);
    }

    printf("\n");
}

void convert_case(char *s, char c)
{
    if (s == NULL)
    {
        return;
    }

    if (c != 'U' && c != 'L')
    {
        printf("Invalid conversion option (use 'U' or 'L')!\n");
        return;
    }

    size_t s_length = strlen(s);
    char temp[s_length + 1u];

    for (size_t i = 0u; i < s_length; i += 1u)
    {
        if (c == 'U')
        {
            temp[i] = (char) toupper((unsigned char) s[i]);
        }
        else if (c == 'L')
        {
            temp[i] = (char) tolower((unsigned char) s[i]);
        }
    }

    temp[s_length] = '\0';
    printf("%s\n", temp);

    // Other ctype functions:
    // isalnum()
    // isalpha()
    // isdigit()
    // isxdigit()
    // isblank()
    // isspace()
    // iscntrl()
    // isgraph()
    // isprint()
```

```
    // ispunct()
    // isupper()
    // islower()
    // ...
}

void swap_array(int *d, size_t j)
{
    int temp = d[j];
    d[j] = d[j + 1u];
    d[j + 1u] = temp;
}
```

9. Pointers & Static

```
// printf:
#include <stdio.h>

void count(void);

int main(void)
{
    // Pointers with variables: ↓
    int var = 10;

    // Pointer to variable:
    int *ptr1 = &var;

    // Modify value:
    *ptr1 = 20;

    // Modify address:
    // ptr1 = &var;

    printf("Value: %d\n", *ptr1);
    // printf("Value: %d\n", var);

    printf("Address: %p\n", ptr1);
    // printf("Address: %p\n", &var);

    // 64-bit system = 8 bytes; 32-bit system = 4 bytes:
    printf("Size: %zu\n", sizeof(ptr1));

    printf("\n");

    // ------------------------------------------------------------

    // Pointers with arrays: ↓
    int arr[] = {1, 2, 3, 4, 5,};

    // ptr2 points to arr[0u]:
    int *ptr2 = arr;
    printf("Value: %d\n", *ptr2);
    printf("Address: %p\n", ptr2);

    // 64-bit system = 8 bytes; 32-bit system = 4 bytes:
    printf("Size: %zu\n", sizeof(ptr2));

    // Change pointer to the next element:
    ptr2 += 1;
```

```
    printf("%d\n", *ptr2);

    // Change pointer to the previous element:
    ptr2 -= 1;
    printf("%d\n", *ptr2);

    // Access the next element without changing the pointer:
    int element = *(ptr2 + 1);
    printf("%d\n", element);

    printf("\n");

    // ------------------------------------------------------------

    // static:
    count();
    count();
    count();

    return 0;
}

void count(void)
{
    // Keep value between calls:
    static int num = 0;

    num += 1;
    printf("%d\n", num);
}
```

10. Memory Management

```
// printf:
#include <stdio.h>

// malloc, calloc, realloc, free:
#include <stdlib.h>

// NULL:
#include <stddef.h>

int main(void)
{
    // Allocate memory for an array of 10 integers (uninitialized):
    int *ptr1 = malloc(10 * sizeof(*ptr1));

    // Check if memory allocation failed:
    if (ptr1 == NULL)
    {
        printf("Malloc failed!\n");
        return 1;
    }

    // ------------------------------------------------------------

    // Allocate memory for an array of 10 integers (zero-initialized):
    int *ptr2 = calloc(10, sizeof(*ptr2));

    // Check if memory allocation failed:
```

```
    if (ptr2 == NULL)
    {
        printf("Calloc failed!\n");
        free(ptr1);
        return 1;
    }

    // ------------------------------------------------------------

    // Resize the first array from 10 integers to 5 integers:
    int *temp = realloc(ptr1, 5 * sizeof(*temp));

    // Check whether reallocation failed:
    if (temp == NULL)
    {
        printf("Realloc failed!\n");
        free(ptr1);
        free(ptr2);
        return 1;
    }

    // temp and ptr1 now point to the same memory:
    ptr1 = temp;

    // ------------------------------------------------------------

    // Free all allocated memory:
    free(ptr1);
    free(ptr2);

    return 0;
}
```

11. Structures

```
// printf:
#include <stdio.h>

// size_t:
#include <stddef.h>

// Structure:
typedef struct Student
{
    char name[8u + 1u];
    int number;
} Student;

// Nested structure:
typedef struct Classroom
{
    char name[13u + 1u];
    Student student;
} Classroom;

int main(void)
{
    // Structures: ↓
    Student s1 = {"Nemanja", 3,};
    printf("%s %d\n", s1.name, s1.number);
```

```
    Student s2 = {"Studenko", 5,};
    Student *ptr = &s2;
    printf("%s %d\n", ptr -> name, ptr -> number);

    // Size of structure:
    printf("Size: %zu bytes\n", sizeof(Student));

    printf("\n");

    // ------------------------------------------------------------

    // Array of structures:
    Student students[] = {s1, s2,};

    size_t students_size = sizeof(students) / sizeof(students[0u]);

    for (size_t i = 0u; i < students_size; i += 1u)
    {
        printf("%s %d\n", students[i].name, students[i].number);
    }

    printf("\n");

    // ------------------------------------------------------------

    // Nested structure:
    Classroom c1 = {"C Programming", s1,};

    printf("%s\n", c1.name);
    printf("%s %d\n", c1.student.name, c1.student.number);

    return 0;
}
```

12. Unions

```
// printf:
#include <stdio.h>

// strcpy:
#include <string.h>

typedef union Data
{
    int i;
    double d;
    char s[6u + 1u];
} Data;

int main(void)
{
    Data data;

    data.i = 10;
    printf("Int: %d\n", data.i);

    data.d = 9.9;
    printf("Double: %.2lf\n", data.d);
```

```
    strcpy(data.s, "Hello!");
    printf("String: %s\n", data.s);

    return 0;
}
```

13. Enumerations

```
// printf:
#include <stdio.h>

typedef enum Week
{
    Sun,
    Mon,
    Tue,
    Wed,
    Thu,
    Fri,
    Sat,
} Week;

int main(void)
{
    Week day = Wed;
    printf("%d\n", day);

    return 0;
}
```

14. Files

```
// printf, FILE, fopen, fclose, fprintf, fgets, ferror, EOF, remove:
#include <stdio.h>

// NULL:
#include <stddef.h>

// exit:
#include <stdlib.h>

int main(void)
{
    FILE *fp;
    char text[1024u + 1u];

    // ------------------------------------------------------------

    // File write: ↓
    fp = fopen("example.txt", "w");

    if (fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }

    if (fprintf(fp, "Hello world!\n") < 0)
    {
```

```
    printf("Error writing to file!\n");
    fclose(fp);
    return 1;
}

if (fclose(fp) == EOF)
{
    printf("Error closing file!\n");
    return 1;
}

// ------------------------------------------------------------

// File append: ↓
fp = fopen("example.txt", "a");

if (fp == NULL)
{
    printf("Error opening file!\n");
    return 1;
}

if (fprintf(fp, "This line is appended\n") < 0)
{
    printf("Error appending to file!\n");
    fclose(fp);
    return 1;
}

if (fclose(fp) == EOF)
{
    printf("Error closing file!\n");
    return 1;
}

// ------------------------------------------------------------

// File read: ↓
fp = fopen("example.txt", "r");

if (fp == NULL)
{
    printf("Error opening file!\n");
    return 1;
}

while (fgets(text, sizeof(text), fp) != NULL)
{
    printf("%s", text);
}

if (ferror(fp))
{
    printf("Error reading file!\n");
    fclose(fp);
    return 1;
}

if (fclose(fp) == EOF)
{
    printf("Error closing file!\n");
```

```
        return 1;
    }

    // ------------------------------------------------------------

    // File remove: ↓
    if (remove("example.txt") != 0)
    {
        printf("Error removing file!\n");
        exit(1);
    }

    return 0;
}
```

15. Math Functions

```
// printf:
#include <stdio.h>

// Math functions:
#include <math.h>

int main(void)
{
    printf("ceil(4.3) = %.2lf\n", ceil(4.3));
    printf("floor(4.8) = %.2lf\n", floor(4.8));

    printf("fabs(-25.0) = %.2lf\n", fabs(-25.0));
    printf("sqrt(25) = %.2lf\n", sqrt(25));
    printf("cbrt(8) = %.2lf\n", cbrt(8));

    printf("pow(5, 2) = %.2lf\n", pow(5, 2));
    printf("exp(1) = %.2lf\n", exp(1));

    printf("fmod(10, 3) = %.2lf\n", fmod(10, 3));

    printf("log(2.71828) = %.2lf\n", log(2.71828));
    printf("log10(100) = %.2lf\n", log10(100));

    printf("cos(0) = %.2lf\n", cos(0));
    printf("sin(0) = %.2lf\n", sin(0));
    printf("tan(0) = %.2lf\n", tan(0));

    return 0;
}
```

16. Random

```
// printf:
#include <stdio.h>

// srand, rand:
#include <stdlib.h>

// time:
#include <time.h>

// NULL:
```

```c
#include <stddef.h>

int main(void)
{
    srand((unsigned int) time(NULL));

    int min = 10;
    int max = 90;

    // Random number: 10-90 (inclusive):
    int range = max - min + 1;

    // rand() % range may introduce modulo bias:
    int number = rand() % range + min;

    printf("Random number: %d\n", number);

    return 0;
}
```

17. Date-Time & Sleep

```c
// printf, perror:
#include <stdio.h>

// time_t, time, struct tm, localtime:
#include <time.h>

// sleep (POSIX - Linux, macOS):
#include <unistd.h>

// NULL:
#include <stddef.h>

int main(void)
{
    char days[7u][9u + 1u] =
    {
        "Sunday",
        "Monday",
        "Tuesday",
        "Wednesday",
        "Thursday",
        "Friday",
        "Saturday",
    };

    while (1)
    {
        time_t now = time(NULL);

        if (now == (time_t) -1)
        {
            perror("Time error!");
            return 1;
        }

        // -------------------------------------------------------

        struct tm *t = localtime(&now);
```

```
        if (t == NULL)
        {
            perror("Localtime error!");
            return 1;
        }

        // ------------------------------------------------------------

        // Date:
        int day_of_week = t -> tm_wday;
        int day = t -> tm_mday;
        int month = t -> tm_mon + 1;
        int year = t -> tm_year + 1900;

        // Time:
        int hour = t -> tm_hour;
        int minute = t -> tm_min;
        int second = t -> tm_sec;

        // ------------------------------------------------------------

        // Thursday, 01.01.2026. - 09:30:00
        printf("%s, %02d.%02d.%04d. - %02d:%02d:%02d\n",
               days[day_of_week], day, month, year,
               hour, minute, second);

        // Sleep 1 second:
        sleep(1u);
    }

    return 0;
}
```

18. Preprocessor Directives

```
// File "myheader.h"

// Protection against being included multiple times:
#ifndef MYHEADER_H
#define MYHEADER_H

#define PI 3.14159

void say_hello(void);

#endif
```

```
// File "myheader.c"

// printf:
#include <stdio.h>

// Include custom header:
#include "myheader.h"

// #define - Define macros:
#define SQUARE(x) ((x) * (x))
```

```
#define MAX(a,b) ((a) > (b) ? (a) : (b))
#define DEBUG

// #ifdef - Check if DEBUG is defined:
#ifdef DEBUG
    #define MESSAGE "Debug mode is ON"
#else
    #define MESSAGE "Debug mode is OFF"
#endif

// #ifndef - Define BUFFER_SIZE if not defined:
#ifndef BUFFER_SIZE
    #define BUFFER_SIZE 1024u
#endif

// #if, #elif, #else, #endif:
#if BUFFER_SIZE >= 1024u
    #define STATUS "Large buffer"
#elif BUFFER_SIZE >= 512u
    #define STATUS "Medium buffer"
#else
    #define STATUS "Small buffer"
#endif

int main(void)
{
    // Custom header function:
    say_hello();

    printf("\n");

    // ------------------------------------------------------------

    int a = 5;
    int b = 10;

    // Never use side effects (i += 1 and i -= 1) in macros:
    printf("Square: %d\n", SQUARE(a));
    printf("Maximum: %d\n", MAX(a, b));

    printf("\n");

    // ------------------------------------------------------------

    // Using PI:
    printf("PI: %.5lf\n", PI);

    printf("\n");

    // ------------------------------------------------------------

    // Conditional compilation:
    printf("Message: %s\n", MESSAGE);
    printf("Buffer: %u\n", BUFFER_SIZE);
    printf("Status: %s\n", STATUS);

    printf("\n");

    // ------------------------------------------------------------

    // Predefined macros:
```

```
    printf("File: %s\n", __FILE__);
    printf("Line: %d\n", __LINE__);
    printf("Date: %s\n", __DATE__);
    printf("Time: %s\n", __TIME__);

    // ------------------------------------------------------------

    // #undef - Remove a macro:
    #undef PI

    return 0;
}

void say_hello(void)
{
    printf("Hello from myheader.h!\n");
}

// #error - Force compilation error:
// #error "Compilation stopped!"
```

19. Argc & Argv

```
// printf:
#include <stdio.h>

int main(int argc, char **argv)
{
    if (argc > 1)
    {
        for (int i = 1; i < argc; i += 1)
        {
            printf("Argument %d: %s\n", i, argv[i]);
        }
    }

    else if (argc < 2)
    {
        printf("Usage: %s <argument>\n", (argc > 0) ? argv[0u] : "program_c");
    }

    return 0;
}
```

20. Multithreading

```
// printf:
#include <stdio.h>

// pthread_t, pthread_create, pthread_join (POSIX - Linux, macOS):
#include <pthread.h>

// NULL:
#include <stddef.h>

void *thread_function(void *arg)
{
    int *ptr = (int *) arg;
    int thread_id = *ptr;
```

```
    for (int i = 0; i < 5; i += 1)
    {
        printf("Thread %d: %d\n", thread_id, (i + 1));
    }

    return NULL;
}

int main(void)
{
    pthread_t thread1;
    pthread_t thread2;

    int id1 = 1;
    int id2 = 2;

    // Create first thread:
    int result1 = pthread_create(
        &thread1, NULL, thread_function, &id1
    );

    if (result1 != 0)
    {
        printf("Failed to create thread 1!\n");
        return 1;
    }

    // Create second thread:
    int result2 = pthread_create(
        &thread2, NULL, thread_function, &id2
    );

    if (result2 != 0)
    {
        printf("Failed to create thread 2!\n");

        // Wait for the first thread before exiting:
        if (pthread_join(thread1, NULL) != 0)
        {
            printf("Failed to join thread 1!\n");
        }

        return 1;
    }

    // Wait for both threads:
    int join1 = pthread_join(thread1, NULL);
    int join2 = pthread_join(thread2, NULL);

    if (join1 != 0 || join2 != 0)
    {
        printf("Failed to join one or both threads!\n");
        return 1;
    }

    printf("Both threads finished\n");

    return 0;
}
```

21. Cool Print

```
// printf, fflush, stdout:
#include <stdio.h>

// sleep (POSIX - Linux, macOS):
#include <unistd.h>

void screen_controls(void)
{
    // Clear entire screen:
    printf("\x1b[2J");

    // Move cursor to top-left:
    printf("\x1b[H");

    // Clear from cursor to end:
    printf("\x1b[0J");

    // Clear from beginning to cursor:
    printf("\x1b[1J");

    // Clear entire current line:
    printf("\x1b[2K");

    // Clear to end of line:
    printf("\x1b[0K");

    // Clear from beginning of line:
    printf("\x1b[1K");

    // ------------------------------------------------------------

    // Move to row 5, column 10:
    printf("\x1b[5;10H");

    // Move up 3 lines:
    printf("\x1b[3A");

    // Move down 3 lines:
    printf("\x1b[3B");

    // Move right 5 columns:
    printf("\x1b[5C");

    // Move left 5 columns:
    printf("\x1b[5D");

    // ------------------------------------------------------------

    // Save cursor position:
    printf("\x1b[s");

    // Move cursor:
    printf("\x1b[5;10H");

    // Restore cursor position:
    printf("\x1b[u");
}

void foreground_colors(void)
```

```
{
    printf("\n------ FOREGROUND COLORS ------\n");

    printf("\x1b[30mBlack\x1b[0m\n");
    printf("\x1b[31mRed\x1b[0m\n");
    printf("\x1b[32mGreen\x1b[0m\n");
    printf("\x1b[33mYellow\x1b[0m\n");
    printf("\x1b[34mBlue\x1b[0m\n");
    printf("\x1b[35mMagenta\x1b[0m\n");
    printf("\x1b[36mCyan\x1b[0m\n");
    printf("\x1b[37mWhite\x1b[0m\n");
}

void background_colors(void)
{
    printf("\n------ BACKGROUND COLORS ------\n");

    printf("\x1b[40m Black background \x1b[0m\n");
    printf("\x1b[41m Red background \x1b[0m\n");
    printf("\x1b[42m Green background \x1b[0m\n");
    printf("\x1b[43m Yellow background \x1b[0m\n");
    printf("\x1b[44m Blue background \x1b[0m\n");
    printf("\x1b[45m Magenta background \x1b[0m\n");
    printf("\x1b[46m Cyan background \x1b[0m\n");
    printf("\x1b[47m White background \x1b[0m\n");
}

void text_formatting(void)
{
    printf("\n------ TEXT FORMATTING ------\n");

    printf("\x1b[0mNormal text\x1b[0m\n");
    printf("\x1b[1mBold text\x1b[0m\n");
    printf("\x1b[2mDim text\x1b[0m\n");
    printf("\x1b[3mItalic text\x1b[0m\n");
    printf("\x1b[4mUnderlined text\x1b[0m\n");
    printf("\x1b[5mBlinking text\x1b[0m\n");
    printf("\x1b[7mInverted colors\x1b[0m\n");
    printf("\x1b[8mHidden text\x1b[0m\n");
    printf("\x1b[9mStrikethrough text\x1b[0m\n");
}

void advanced_colors(void)
{
    printf("\n------ ADVANCED COLORS ------\n");

    // 256-color foreground:
    printf("\x1b[38;5;196m256-color red\x1b[0m\n");
    printf("\x1b[38;5;208m256-color orange\x1b[0m\n");
    printf("\x1b[38;5;46m256-color green\x1b[0m\n");

    // RGB true color foreground:
    printf("\x1b[38;2;255;100;50mRGB orange text\x1b[0m\n");
    printf("\x1b[38;2;255;0;255mRGB pink text\x1b[0m\n");
    printf("\x1b[38;2;0;255;255mRGB cyan text\x1b[0m\n");

    // RGB true color background:
    printf("\x1b[48;2;0;100;255m RGB blue background \x1b[0m\n");
    printf("\x1b[48;2;255;100;50m RGB orange background \x1b[0m\n");
}
```

```
void color_combinations(void)
{
    printf("\n------ COLOR COMBINATIONS ------\n");

    // Bold + blink + red background + white text:
    printf("\x1b[1;5;41;37m WARNING \x1b[0m\n");

    // Bold + green background + black text:
    printf("\x1b[1;42;30m SUCCESS \x1b[0m\n");

    // Underline + blue background + yellow text:
    printf("\x1b[4;44;33m INFO \x1b[0m\n");

    // Bold + magenta background + white text:
    printf("\x1b[1;45;37m NOTICE \x1b[0m\n");

    // Other combinations: ↓
    printf("\x1b[1;33mYellow text\x1b[0m\n");
    printf("\x1b[35mPink / Magenta text\x1b[0m\n");
    printf("\x1b[36mLight cyan text\x1b[0m\n");

    printf("\x1b[43m Yellow background \x1b[0m\n");
    printf("\x1b[45m Pink background \x1b[0m\n");
    printf("\x1b[46m Cyan background \x1b[0m\n");
}

void colored_blocks(void)
{
    printf("\n------ COLORED BLOCKS ------\n");

    printf("\x1b[41m      \x1b[0m Red block\n");
    printf("\x1b[44m      \x1b[0m Blue block\n");
    printf("\x1b[47m      \x1b[0m White block\n");
    printf("\x1b[42m      \x1b[0m Green block\n");
    printf("\x1b[43m      \x1b[0m Yellow block\n");
    printf("\x1b[45m      \x1b[0m Magenta block\n");
    printf("\x1b[46m      \x1b[0m Cyan block\n");
}

void terminal_controls(void)
{
    printf("\n------ TERMINAL CONTROLS ------\n");

    // Hide cursor:
    printf("\x1b[?25l");
    fflush(stdout);
    sleep(1u);

    // Show cursor:
    printf("\x1b[?25h");
    fflush(stdout);

    // Save cursor position:
    printf("\x1b[s");
    printf("Cursor position saved\n");
    fflush(stdout);
    sleep(1u);

    // Restore cursor position:
    printf("\x1b[u");
    fflush(stdout);
```

```
    // Request cursor position (terminal response required):
    // printf("\x1b[6n");

    // Enter alternate screen:
    // printf("\x1b[?1049h");

    // Leave alternate screen:
    // printf("\x1b[?1049l");
}

int main(void)
{
    // Clear terminal and move to top-left:
    printf("\x1b[2J\x1b[H");
    fflush(stdout);

    // screen_controls();
    // fflush(stdout);

    // Header:
    printf("\x1b[1;36m");
    printf("----------------------------------------------------------------\n");
    printf("      ANSI ESCAPE SEQUENCE DEMO      \n");
    printf("----------------------------------------------------------------\n");
    printf("\x1b[0m");

    // Display all demonstrations:
    foreground_colors();
    background_colors();
    text_formatting();
    advanced_colors();
    color_combinations();
    colored_blocks();
    terminal_controls();

    // Final message:
    printf("\n");
    printf("\x1b[1;32m");
    printf("----------------------------------------------------------------\n");
    printf("      DEMONSTRATION COMPLETE      \n");
    printf("----------------------------------------------------------------\n");
    printf("\x1b[0m");
    fflush(stdout);

    return 0;
}
```

22. Execution Time

```
// printf:
#include <stdio.h>

// clock_t, clock, CLOCKS_PER_SEC:
#include <time.h>

int main(void)
{
    clock_t start = clock();
    if (start == (clock_t) -1)
```

```
    {
        printf("Unable to determine processor time!\n");
        return 1;
    }

    // Code whose execution time is being measured:
    for (int i = 0; i < 100000; i += 1)
    {
        printf("%d ", i);
    }
    printf("\n");

    clock_t end = clock();
    if (end == (clock_t) -1)
    {
        printf("Unable to determine processor time!\n");
        return 1;
    }

    double cpu_time = (double) (end - start) / CLOCKS_PER_SEC;
    printf("CPU time: %.6lf seconds\n", cpu_time);

    return 0;
}
```

23. Bonus: Linux

```
// printf:
#include <stdio.h>

// system:
#include <stdlib.h>

int main(void)
{
    if (system("cd / && ls") != 0)
    {
        printf("System error!\n");
        return 1;
    }

    return 0;
}
```